

Cómo gestionar el contexto profesionalmente con Claude Code y ChatGPT en proyectos de meses

La idea más importante de todo este informe es ésta:

**Un desarrollador que trabaja bien con IA no intenta que el modelo “recuerde todo”.
Diseña un sistema para que el modelo pueda reconstruir rápidamente lo que necesita.**

Ése es el cambio de mentalidad fundamental.

La ventana de contexto no debe entenderse como una base de datos, ni como memoria permanente, ni como el lugar donde guardar el conocimiento de un proyecto. Es más parecido a la **mesa de trabajo actual** del modelo. El repositorio, la documentación, los archivos de decisiones, las especificaciones y los tests son el verdadero almacenamiento duradero.

Anthropic lo expresa de forma muy parecida al describir los problemas de agentes que trabajan durante muchas ventanas de contexto: las sesiones tienen que dejar artefactos claros para que la siguiente pueda reconstruir rápidamente el estado del trabajo. En sus experimentos con agentes de larga duración, Anthropic acabó usando historial Git, archivos de progreso y especificaciones estructuradas precisamente porque la compactación y la conversación por sí solas no bastaban. ¹

La arquitectura mental que te recomiendo usar es:

Concepto	Analogía útil	En un proyecto real
Pesos del modelo	Conocimientos adquiridos antes de conocerle	Programación general, Python, React, patrones
Ventana de contexto	Mesa de trabajo / RAM	Lo que Claude está considerando ahora
Conversación	Cuaderno de trabajo temporal	Decisiones, preguntas, intentos, debugging
Compactación/ resumen	Informe de relevo	Estado condensado de una sesión larga
<code>CLAUDE.md</code>	Manual de incorporación del equipo	Reglas permanentes del repositorio
Skills	Manuales especializados	Deploy, API design, testing, migrations
Subagentes	Compañeros con su propia mesa	Investigación, revisión, exploración

Concepto	Analogía útil	En un proyecto real
Repositorio	Disco duro / fuente de verdad	Código, docs, tests, Git
Documentación	Memoria institucional	Arquitectura, ADR, requisitos, decisiones
Memoria de ChatGPT	Notas persistentes sobre ti/proyecto	Preferencias, información reutilizable
Proyecto de ChatGPT	Espacio de trabajo contextual	Chats + archivos + instrucciones

La consecuencia práctica es decisiva:

no diseñes tu forma de trabajar alrededor de conversaciones infinitas. Diseña tu proyecto para sobrevivir perfectamente a que mañana abras una conversación completamente nueva.

La ventana de contexto y lo que realmente significa “recordar”

Una ventana de contexto es la cantidad máxima de información que un modelo puede tener disponible durante una inferencia o secuencia de trabajo. Se mide en **tokens**, no directamente en palabras. Anthropic la describe explícitamente como la “working memory” del modelo, diferenciándola de los conocimientos aprendidos durante el entrenamiento. ²

A nivel conceptual, un modelo Transformer recibe una secuencia de tokens y utiliza mecanismos de atención para relacionar unas partes de esa secuencia con otras mientras genera nuevos tokens. Ésta es la idea introducida por la arquitectura Transformer original. ³

Una simplificación razonable sería imaginar:

CONTEXTO ACTUAL

[reglas del sistema]
[instrucciones]
[documentación relevante]
[conversación anterior]
[archivos leídos]
[resultados de herramientas]
[tu última petición]

↓

MODELO

↓

siguiente respuesta

Lo importante es que **todo eso compite por espacio**.

En Claude Code la documentación oficial es especialmente explícita: la ventana puede contener la conversación, contenido de archivos, resultados de comandos, `CLAUDE.md`, auto memory, Skills cargadas e instrucciones del sistema. ⁴

Por ejemplo, imagina una sesión:

Sistema	10k
CLAUDE.md	4k
Herramientas	5k
Conversación	20k
Lectura de archivos	35k
Logs de tests	25k
Investigación	40k
Intentos fallidos	30k
Última implementación	25k

El problema no es únicamente alcanzar un límite duro.

Hay **tres formas distintas de “perder contexto”**.

Pérdida física. Cierta información deja de entrar en la ventana porque se elimina, trunca o sustituye.

Pérdida por compresión. Un historial de 50 páginas puede convertirse en un resumen de dos páginas. La idea general permanece, pero determinados detalles desaparecen.

Pérdida de atención efectiva. La información sigue técnicamente en contexto, pero el modelo no la utiliza bien.

Este último fenómeno es muy importante. Anthropic advierte que disponer de más contexto no implica automáticamente mejores resultados y habla de *context rot*: la precisión y el recuerdo pueden degradarse conforme aumenta el contexto. ⁵

La literatura académica ha encontrado además el fenómeno llamado **“lost in the middle”**: en determinados experimentos, los modelos recuperan mejor información situada al principio o al final de grandes contextos que información relevante enterrada en posiciones intermedias. ⁶

Por eso:

Una ventana de un millón de tokens no equivale a un millón de tokens de atención perfecta.

Claude Code dispone actualmente de configuraciones/modelos con ventanas muy grandes, incluidas variantes de alrededor de un millón de tokens, pero Anthropic sigue recomendando gestionar activamente el contexto; incluso su documentación de ingeniería argumenta que ventanas mayores no eliminan los problemas de contaminación, relevancia y degradación. ⁷

Qué ocurre cuando Claude Code lleva mucho tiempo trabajando

Claude Code no espera necesariamente a chocar contra el límite.

Dispone de **auto-compaction**: cuando el contexto se aproxima a su límite, gestiona automáticamente el historial. La documentación actual describe un comportamiento especialmente interesante: primero puede eliminar resultados antiguos de herramientas y, si sigue necesitando espacio, resume la

conversación. Las instrucciones detalladas de etapas anteriores pueden perderse durante este proceso.

8

Es decir:

ANTES

Prompt inicial

↓

Exploración

↓

archivo A completo

↓

archivo B completo

↓

resultado enorme de npm test

↓

debugging intento 1

↓

debugging intento 2

↓

decisión arquitectura

↓

implementación

↓

más logs

puede acabar conceptualmente parecido a:

DESPUÉS DE COMPACTAR

Resumen:

- estamos implementando autenticación
- se decidió usar JWT
- archivos modificados X, Y, Z
- pruebas actuales...
- siguiente tarea...

+ mensajes recientes

+ CLAUDE.md

+ contexto actual

Esto permite continuar, pero evidentemente:

50.000 tokens originales

↓

resumen

↓
5.000 tokens importantes

no puede ser una transformación sin pérdida.

Anthropic reconoce precisamente que la compactación no siempre transmite perfectamente el estado a la siguiente etapa o sesión, motivo por el que recomienda complementar la conversación con artefactos externos. ⁹

Entonces, ¿Claude “lee todo el chat” cada vez?

Para **Claude Code**, el modelo mental correcto es: el modelo procesa el **contexto activo que Claude Code ha construido para esa petición**, que puede contener gran parte del historial más otros elementos, pero después de compactaciones ya no equivale necesariamente al transcript original íntegro. ⁸

Para **ChatGPT**, sería imprudente afirmar que cada turno consiste literalmente en reenviar siempre todo el transcript bruto. OpenAI no documenta su arquitectura interna con ese nivel de detalle en la interfaz ChatGPT y actualmente existen mecanismos adicionales de memoria, archivos, historial referenciado y recuperación entre conversaciones. El modelo mental más robusto es pensar que ChatGPT construye un **working context** con la conversación y las fuentes relevantes disponibles, no que “relee una cinta infinita desde el primer mensaje”. Esto último es una inferencia basada en las funciones de Memory y Projects documentadas por OpenAI, no una descripción publicada de su implementación interna. ¹⁰

ChatGPT: conversación, memoria y Proyectos son cosas diferentes

Aquí suele producirse una de las mayores confusiones de los principiantes.

Hay que separar:

VENTANA DE CONTEXTO
≠
HISTORIAL DEL CHAT
≠
MEMORIA
≠
ARCHIVOS DEL PROYECTO

La conversación actual contiene el trabajo realizado en ese hilo.

La memoria de ChatGPT puede conservar información útil más allá de la conversación actual. OpenAI distingue actualmente información memorizada y referencia al historial de conversaciones; además advierte explícitamente que ChatGPT **no recuerda todos los detalles de chats anteriores** y recomienda las memorias guardadas para información que quieras que permanezca disponible. ¹¹

Por tanto, llamar a esto “memoria permanente” es una simplificación. Es mejor decir **memoria persistente**: puede sobrevivir entre chats, pero puede actualizarse y el usuario puede editarla o eliminarla. ¹²

Una preferencia como:

“Estoy aprendiendo programación y prefiero que me expliques el porqué antes de darme código.”

puede tener sentido como memoria o personalización.

Pero algo como:

“En el endpoint `/api/orders` decidimos usar idempotency keys porque Stripe puede repetir callbacks y la migration 018 ya añadió `request_id`.”

no debería depender de tu memoria personal de ChatGPT.

Eso pertenece al proyecto.

Idealmente:

```
docs/architecture/payments.md
```

o:

```
docs/adr/004-idempotency-strategy.md
```

Qué aporta un Proyecto de ChatGPT

Los Projects de ChatGPT reúnen conversaciones, archivos e instrucciones en un espacio de trabajo. OpenAI indica que los chats de un proyecto pueden apoyarse en otros chats y archivos del mismo proyecto y que, para usuarios Plus y Pro, ChatGPT prioriza los chats y archivos del proyecto cuando trabaja dentro de él. ¹³

Eso hace posible un patrón mucho mejor que “un chat monstruoso”:

```
PROYECTO: SaaS Facturación
```

```
├─ Chat: arquitectura
├─ Chat: autenticación
├─ Chat: pagos Stripe
├─ Chat: frontend
├─ Chat: testing
├─ Chat: deployment
├─
└─ arquitectura.md
```

```
├─ product-spec.md
├─ api-contract.md
└─ decisions.md
```

Las conversaciones pueden estar especializadas sin quedar totalmente aisladas del entorno conceptual del proyecto.

Además, puedes guardar respuestas importantes como fuentes del proyecto para reutilizarlas posteriormente. ¹⁴

Las instrucciones de Proyecto

Las instrucciones de Proyecto son el equivalente conceptual más cercano, dentro de ChatGPT, a parte del papel que cumple `CLAUDE.md`.

OpenAI señala que sólo se aplican dentro del proyecto y que sustituyen las Custom Instructions globales mientras trabajas dentro de él. ¹⁵

Unas buenas instrucciones para tu caso podrían ser:

Contexto del usuario

Estoy aprendiendo ingeniería de software y desarrollo con IA.
Quiero comprender las decisiones, no limitarme a copiar código.

Forma de trabajar

- Actúa como senior engineer y mentor.
- Antes de cambios importantes, explica arquitectura y trade-offs.
- No inventes el estado del proyecto: consulta los archivos disponibles.
- Distingue hechos del proyecto de hipótesis.
- Para tareas grandes, propón primero una especificación.
- Mantén las soluciones simples salvo que exista una razón clara para aumentar la complejidad.
- Señala deuda técnica y riesgos.
- Incluye estrategia de validación y tests.
- Cuando una decisión deba persistir, recomiéndame documentarla.

Observa que no he metido aquí:

Todos los endpoints
Todo el esquema SQL
Todas las dependencias
Todo el roadmap
Todos los bugs conocidos

Eso sería documentación, no instrucciones.

Project-only memory

Actualmente ChatGPT también permite crear proyectos con **project-only memory**. En ese modo, las memorias guardadas externas no se utilizan; las conversaciones pueden referenciar otras conversaciones del mismo proyecto, pero no conversaciones externas. ¹⁶

Para proyectos profesionales independientes puede ser una opción muy interesante:

```
Proyecto A: aplicación inmobiliaria
  |
  └─ contexto aislado

Proyecto B: curso de Python
  |
  └─ contexto aislado

Proyecto C: SaaS financiero
  |
  └─ contexto aislado
```

Evitas que información de proyectos conceptualmente distintos se mezcle.

Pero hay una distinción crucial:

Project Memory no convierte todos los chats del proyecto en una gigantesca ventana de contexto siempre cargada.

OpenAI habla de poder **referenciar** conversaciones y de priorizar chats y archivos relevantes; debes pensar en ello como acceso contextual, no como una garantía de que absolutamente cada palabra de todo el proyecto esté presente en cada inferencia. ¹⁷

Claude Code cambia radicalmente el modelo porque el repositorio es memoria externa

Claude Code es más interesante para desarrollo de software prolongado porque no depende principalmente de la conversación.

Su bucle de funcionamiento se basa en tres operaciones recurrentes:

```
REUNIR CONTEXTO
  ↓
ACTUAR
  ↓
VERIFICAR
  ↓
REPETIR
```


Claude Code puede buscar archivos, leer código, modificarlo, ejecutar comandos, inspeccionar Git y ejecutar tests. El resultado de las herramientas vuelve al modelo y alimenta la siguiente decisión. ⁸

Esto significa que no necesitas meter tu aplicación completa en contexto.

Supongamos que tienes:

```
mi-app/  
├─ 3.200 archivos  
└─ 400.000 líneas de código
```

La estrategia incorrecta sería:

```
400.000 líneas  
    ↓  
contexto  
    ↓  
modelo
```

Claude Code puede hacer algo conceptualmente mejor:

```
"Corrige el refresh de sesión"  
    ↓  
grep "refreshToken"  
    ↓  
lee auth/service.ts  
    ↓  
lee middleware/session.ts  
    ↓  
consulta tests relacionados  
    ↓  
modifica 3 archivos  
    ↓  
ejecuta tests
```

Sólo necesita cargar una **fracción relevante** del repositorio.

Anthropic describe Claude Code precisamente como una estrategia híbrida: `CLAUDE.md` entra por adelantado, mientras que herramientas como glob y grep permiten recuperar información del entorno justo cuando es necesaria. ¹⁸

Ésta es probablemente la idea más importante para entender cómo trabajan actualmente los agentes de programación:

No hay que introducir todo el conocimiento en la ventana. Hay que proporcionar al agente mecanismos fiables para localizar el conocimiento cuando lo necesite.

`CLAUDE.md` : la memoria institucional mínima

Cada sesión nueva de Claude Code empieza con una ventana de contexto nueva. Para transportar conocimiento entre sesiones, Claude Code utiliza actualmente dos mecanismos principales:

`CLAUDE.md` y auto memory. ¹⁹

Un proyecto profesional podría contener:

```
my-app/  
├─ CLAUDE.md  
├─ README.md  
├─ docs/  
│   ├── architecture.md  
│   ├── product.md  
│   ├── database.md  
│   ├── api.md  
│   ├── decisions/  
│   └─ runbooks/  
├─ src/  
├─ tests/  
└─ .claude/  
    ├── rules/  
    ├── skills/  
    └─ agents/
```

Tu `CLAUDE.md` no debería convertirse en una enciclopedia.

Anthropic recomienda mantenerlo conciso —su guía actual sugiere apuntar a menos de unas 200 líneas por archivo— porque se carga en contexto al comenzar las sesiones y un archivo excesivo consume contexto y puede reducir la adherencia a las propias instrucciones. ²⁰

Ejemplo:

```
# Project  
  
SaaS de facturación para autónomos españoles.  
  
# Stack  
  
- Next.js  
- TypeScript strict  
- PostgreSQL  
- Prisma  
- pnpm
```

```

# Commands

- pnpm dev
- pnpm test
- pnpm lint
- pnpm typecheck

# Architecture

- Route handlers must not contain business logic.
- Business logic lives in src/domain/.
- Database access lives in src/repositories/.

# Rules

- Never use `any`.
- Add tests for bug fixes.
- Run typecheck and relevant tests after changes.
- Do not change database schema without a migration.

# Documentation

Read docs/architecture.md before major architectural changes.
Read docs/database.md before schema changes.

# Compact Instructions

When compacting, preserve:
- current objective
- architectural decisions
- changed files
- failing tests
- unresolved problems
- exact next step

```

Éste sí es un buen uso de `CLAUDE.md`.

Anthropic recomienda escribir ahí las cosas que, de otro modo, tendrías que explicarle de nuevo a Claude: comandos, convenciones, arquitectura y reglas recurrentes. También recomienda trasladar procedimientos más especializados a Skills o reglas específicas por ruta. ¹⁹

Además, los `CLAUDE.md` situados en subdirectorios pueden cargarse cuando Claude trabaja con archivos de esas zonas, y `.claude/rules/` permite establecer instrucciones condicionadas por rutas. Esto es particularmente útil en monorepos. ²¹

Por ejemplo:

```

.claude/rules/
└─ frontend.md

```

```
├─ backend.md
├─ database.md
└─ testing.md
```

donde `database.md` sólo se activa cuando el trabajo toca ciertas rutas.

Eso es **context engineering**: información correcta, en el momento correcto.

Skills: conocimiento bajo demanda

Una Skill no tiene el mismo propósito que `CLAUDE.md`.

Piensa:

```
CLAUDE.md
"Claude debe saber esto siempre"

Skill
"Claude debe conocer esto cuando haga esta clase de trabajo"
```

Anthropic carga normalmente las descripciones de Skills para que Claude conozca su existencia, pero el contenido completo de una Skill se introduce cuando se invoca. ²²

Ejemplos razonables:

```
.claude/skills/
├─ deploy/
│   └─ SKILL.md
├─ create-api-endpoint/
│   └─ SKILL.md
├─ database-migration/
│   └─ SKILL.md
├─ security-review/
│   └─ SKILL.md
└─ release/
    └─ SKILL.md
```

Esto evita que un procedimiento de despliegue de 3.000 tokens esté presente mientras arreglas CSS.

Es importante, sin embargo, no pensar que Skills son mágicamente gratuitas. Cuando se invoca una Skill, su contenido entra en la conversación; Anthropic documenta también cómo las Skills recientes pueden reinyectarse después de una compactación dentro de ciertos presupuestos de tokens. ²³

Subagentes: ventanas de contexto independientes

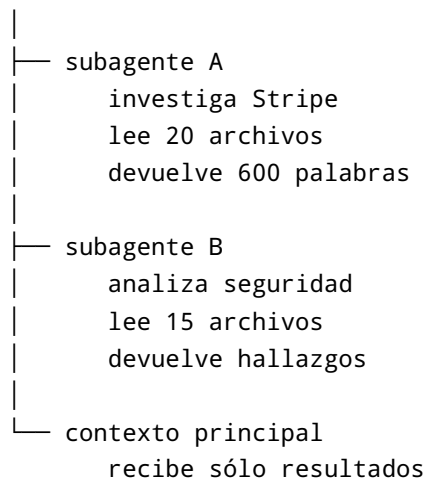
Aquí aparece una de las herramientas más potentes.

Un subagente ejecuta una tarea utilizando **su propia ventana de contexto** y devuelve al agente principal un resultado condensado. Anthropic recomienda utilizarlos precisamente cuando una investigación llenaría el hilo principal de archivos, logs o búsquedas que después ya no serán necesarias. ²⁴

Imagina:

CONTEXTO PRINCIPAL

"Estamos implementando pagos"



Sin subagentes:

```
main context
├─ 20 archivos Stripe
├─ búsquedas
├─ documentación
├─ logs
├─ 15 archivos seguridad
├─ más logs
└─ implementación
```

Con subagentes:

```
main context
├─ objetivo
├─ resumen investigación Stripe
├─ resumen seguridad
└─ implementación
```

Ésta es una reducción enorme del **ruido dentro del contexto principal**, aunque no implica necesariamente menor consumo total de tokens: los subagentes siguen utilizando tokens en sus ventanas independientes. Su principal beneficio es aislamiento y calidad del contexto principal. ²⁵

Auto memory

Claude Code también dispone actualmente de auto memory: notas que Claude puede conservar por repositorio basándose en aprendizajes y correcciones. El `MEMORY.md` inicial tiene límites específicos de carga automática; detalles adicionales pueden almacenarse en archivos temáticos que Claude consulta cuando los necesita. ²⁶

Puedes imaginar:

```
memory/  
├─ MEMORY.md  
├─ debugging.md  
├─ api-conventions.md  
└─ testing-notes.md
```

Pero incluso aquí mi recomendación profesional sería:

si una decisión es parte del producto, escríbela en el repositorio; no dependas exclusivamente de auto memory.

Por ejemplo:

```
"Mis preferencias al trabajar con Claude"  
→ memory / CLAUDE.local.md  
  
"Decidimos que Orders utiliza optimistic locking"  
→ ADR del repositorio  
  
"Para ejecutar tests localmente necesitas Redis"  
→ README / CLAUDE.md / docs  
  
"Encontramos que el bug X ocurre si..."  
→ issue / tests / documentación técnica
```

Cómo organizar chats, sesiones y resúmenes en proyectos largos

La pregunta "¿un único chat enorme o muchos?" tiene una respuesta bastante clara:

Ni un chat por cada pregunta ni un chat único por proyecto. La unidad correcta suele ser una tarea coherente.

Para ChatGPT, usaría:

```
1 Proyecto  
↓  
muchos chats especializados
```

↓
documentación compartida

Para Claude Code:

1 repositorio
↓
muchas sesiones
↓
CLAUDE.md + docs + Git + tests

No mantendría deliberadamente una única sesión durante meses.

Anthropic aconseja explícitamente usar `/clear` entre tareas no relacionadas y señala que sesiones largas llenas de contexto irrelevante pueden degradar el rendimiento. También recomienda que, después de preparar una especificación grande, se pueda empezar una sesión nueva con contexto limpio para implementarla. ²⁷

La unidad natural sería algo como:

Sesión A
Diseñar autenticación

Sesión B
Implementar backend de autenticación

Sesión C
Implementar frontend de login

Sesión D
Tests E2E de autenticación

Sesión E
Security review

No:

Sesión A
enero → febrero → marzo → abril → ...

Cuándo continuar la misma conversación

Continúa cuando el nuevo trabajo depende intensamente del razonamiento reciente.

Por ejemplo:

```
Analizamos bug
→ identificamos causa
→ decidimos solución
→ modificamos código
→ ejecutamos tests
```

Separarlo en cuatro conversaciones sería contraproducente porque cada fase utiliza intensamente el contexto de la anterior.

Cuándo empezar otra

Abre contexto limpio cuando pases de:

```
autenticación
→ sistema de facturación
```

o:

```
brainstorm
→ especificación cerrada
```

o:

```
investigación de arquitectura
→ implementación
```

o cuando la sesión esté contaminada por numerosos intentos fallidos.

La guía actual de Claude Code da incluso una heurística práctica: cuando has corregido al modelo repetidamente por el mismo problema, puede ser mejor limpiar el contexto y empezar otra vez con un prompt mejor formulado que conservar todos los intentos fallidos. ²⁷

Esto tiene sentido porque un historial así:

```
Intento incorrecto A
No hagas A
Intento incorrecto B
No, tampoco B
Prueba C
C casi funciona
Deshaz parte de C
Volvamos parcialmente a A...
```

es un entorno intelectual horrible.

Para una persona también lo sería.

Un contexto nuevo puede empezar simplemente con:

```
Objetivo:
Corregir X.

Diagnóstico confirmado:
Y.

Restricciones:
A, B, C.

Intentos descartados:
D y E por estas razones.

Estado del código:
commit abc123.

Criterio de éxito:
tests X, Y y Z pasan.
```

Eso es muchísimo mejor.

Cuándo compactar o pedir un resumen

No esperaríamos al desastre.

Las señales típicas son:

Señal	Interpretación
Vuelve a preguntar algo decidido	El estado ya no está suficientemente presente
Contradice una decisión anterior	Posible degradación o ruido
Repite investigaciones	Contexto útil perdido o enterrado
Empieza a confundir archivos	Exceso de información
Propone soluciones previamente descartadas	Razones antiguas ya no pesan
Olvida restricciones	Instrucciones enterradas/compactadas
Hay muchísimos logs y tool calls	Contexto contaminado
Has cambiado de tarea	Contexto anterior innecesario
Claude Code <code>/context</code> muestra presión alta	Conviene limpiar/compactar

Claude Code incorpora precisamente `/context` para mostrar qué está ocupando la ventana y `/compact` para resumirla; también permite orientar la compactación con instrucciones como `/compact focus on the API changes`.

Cómo debe ser un resumen realmente útil

No pidas:

“Hazme un resumen de la conversación.”

Pide un **handoff de ingeniería**.

Por ejemplo:

Prepara un handoff técnico para continuar este trabajo en una sesión completamente nueva.

Debe contener:

1. Objetivo del trabajo.
2. Estado actual.
3. Arquitectura relevante.
4. Decisiones tomadas y motivos.
5. Decisiones explícitamente descartadas y motivos.
6. Archivos modificados.
7. Interfaces, schemas o contratos importantes.
8. Bugs o problemas abiertos.
9. Tests ejecutados y resultado.
10. Comandos necesarios para reproducir el entorno.
11. Restricciones que no deben violarse.
12. Próximo paso concreto.
13. Criterio de finalización.

No incluyas conversación irrelevante ni intentos fallidos salvo que expliquen una restricción importante.

El resultado debería parecerse a esto:

Goal

Implement password reset.

Current status

Token generation and persistence are implemented.
Email delivery remains pending.

Decisions

- Tokens expire after 30 minutes.
- Store only SHA-256 token hashes.
- A reset invalidates all previous tokens.

```

# Modified files

- src/auth/reset-service.ts
- src/repositories/reset-token.ts
- prisma/schema.prisma
- tests/auth/reset.test.ts

# Verification

Passing:
- pnpm test auth/reset
- pnpm typecheck

Failing:
- email integration test because Mailpit isn't configured.

# Important constraints

Do not expose raw reset tokens in logs.
Do not change the Session table.

# Next step

Implement EmailProvider.sendPasswordReset().

# Definition of done

All reset tests pass and an E2E request successfully changes
the password and invalidates the token.

```

Eso es un resumen que **puede reemplazar una conversación**.

Y ése debe ser tu objetivo.

ChatGPT frente a Claude Code: dónde vive realmente el contexto

Aquí está la comparación que considero más útil para tu caso.

Dimensión	ChatGPT	Claude Code
Unidad principal	Conversación/Proyecto	Repositorio/sesión
Código	Normalmente proporcionado o accesible mediante herramientas/archivos	Acceso directo al workspace
Persistencia principal	Memory + Projects + archivos	Archivos + Git + CLAUDE.md + auto memory
Recuperación de código	Dependiente del entorno/herramientas	Grep, Glob, Read, code intelligence

Dimensión	ChatGPT	Claude Code
Instrucciones de proyecto	Project Instructions	<code>CLAUDE.md</code> , rules
Procedimientos reutilizables	Instrucciones/GPTs/herramientas según entorno	Skills
Trabajo aislado	Depende de herramientas/agentes disponibles	Subagents con contexto independiente
Compactación explícita	Gestionada por producto	Auto-compaction + <code>/compact</code>
Inspección de consumo	Menos transparente en UI	<code>/context</code>
Reinicio deliberado	Chat nuevo	<code>/clear</code> o sesión nueva
Fuente de verdad ideal	Archivos del proyecto/repositorio	Repositorio
Caso fuerte	Pensamiento, mentoría, investigación, diseño	Implementación agentic sobre código real

Claude Code tiene una ventaja conceptual muy importante para ingeniería: **el código no necesita residir permanentemente en la ventana de contexto para seguir existiendo para Claude.**

Está en disco.

Claude puede volver a leerlo.

Lo mismo ocurre con Git.

Si en una sesión anterior Claude implementó:

```
commit e19ac4
feat(auth): add refresh token rotation
```

la siguiente sesión no necesita recordar cada frase utilizada durante la implementación. Puede inspeccionar:

```
git log
git show e19ac4
```

y leer el código actual.

Esto transforma la ecuación de:

```
MEMORIA = conversación
```

en:

```
ESTADO REAL
=
código
+ git
+ tests
+ documentación
+ specs
+ configuración
```

y:

```
CONTEXTO DEL MODELO
=
subconjunto relevante del estado real
+ objetivo actual
```

Ése es un modelo muchísimo más robusto.

La documentación no sólo ayuda a humanos

En desarrollo asistido por agentes, la documentación se convierte en una **API cognitiva para la IA**.

Un repositorio así:

```
docs/
├─ architecture.md
├─ domain-model.md
├─ authentication.md
├─ database.md
├─ api-contracts.md
├─ deployment.md
├─ testing.md
└─ adr/
   └─ 001-use-postgres.md
      └─ 002-auth-strategy.md
         └─ 003-event-processing.md
```

permite al agente hacer:

```
necesito modificar auth
↓
```

```
lee CLAUDE.md
  ↓
descubre docs/authentication.md
  ↓
lee ADR relevante
  ↓
busca implementación
```

en vez de necesitar:

"¿Qué decidimos hace 47.000 tokens?"

Y aquí aparece una paradoja importante

Muchos principiantes piensan:

"Cuanto más contexto le dé, mejor."

Los desarrolladores con experiencia tienden a pensar:

"¿Cuál es el contexto mínimo que permite tomar correctamente esta decisión?"

Anthropic formula su enfoque de context engineering alrededor precisamente de esa selección activa del contexto y advierte contra contaminación y material irrelevante. ²⁹

Por tanto:

MÁS CONTEXTO

no necesariamente significa:

MEJOR CONTEXTO

Lo que buscas es:

```
CONTEXTO
relevante
actual
no contradictorio
fácil de recuperar
suficiente para verificar
```

Las prácticas profesionales que te recomiendo adoptar desde el principio

Tu sistema debería estar construido alrededor de una sola regla:

Toda información crítica debe tener un hogar persistente fuera del historial conversacional.

No confíes en que “Claude lo sabe porque se lo dije el martes”.

Usa esta jerarquía de almacenamiento

Cuando aparezca información nueva, pregúntate dónde debería vivir:

Información	Lugar
Preferencia personal	Memory / instrucciones personales
Regla de todos tus proyectos	instrucciones globales
Regla permanente del repo	CLAUDE.md
Regla sólo de backend	.claude/rules/backend.md
Procedimiento repetible	Skill
Decisión arquitectónica	ADR
Especificación de funcionalidad	docs/specs/...
Estado actual	Git + issue/task
Bug conocido	issue
Comportamiento esperado	test
API	OpenAPI / docs
Modelo de datos	schema + docs
Investigación temporal	conversación
Investigación que importa después	documento
Próximo trabajo	issue / TODO estructurado
Lección descubierta por Claude	auto memory o docs según importancia

Observa especialmente:

Un test es una forma excelente de memoria.

Supongamos que el contexto original decía:

“Cuando un usuario cancela una suscripción, debe mantener acceso hasta el final del periodo facturado.”

Meses después, puedes perder la conversación.

Pero si existe:

```
it("keeps access until the end of the paid billing period", ...)
```

la decisión se ha convertido en código ejecutable.

El agente no necesita recordar la conversación.

Usa especificaciones antes de funcionalidades grandes

Anthropic recomienda explícitamente, para funcionalidades grandes, realizar la fase de definición primero, producir una especificación autocontenida y después comenzar una sesión limpia para ejecutarla. ²⁷

Tu flujo podría ser:

```
idea
↓
entrevista/requisitos
↓
SPEC.md
↓
revisión
↓
sesión limpia
↓
plan
↓
implementación
↓
tests
↓
documentación
↓
commit
```

Ejemplo:

```
docs/specs/
└─ password-reset.md
```

con:


```
# Problem

# User stories

# Scope

# Out of scope

# UX

# API

# Data model

# Security

# Error handling

# Tests

# Acceptance criteria
```

Esto permite destruir voluntariamente todo el contexto de brainstorming.

Ya no lo necesitas.

Termina cada sesión dejando el repositorio “handoff-ready”

Anthropic llegó a esta conclusión en su trabajo sobre agentes de larga duración: cada sesión debe realizar progreso incremental y dejar el entorno en un estado limpio y comprensible para la siguiente.

1

Antes de terminar una sesión importante, pide:

Antes de cerrar:

- ejecuta los tests relevantes;
- revisa git diff;
- identifica trabajo incompleto;
- actualiza documentación si ha cambiado el comportamiento;
- registra decisiones arquitectónicas nuevas;
- deja claro el siguiente paso;
- no dejes código parcialmente roto salvo que esté documentado.

Idealmente:

```
git status
```

debería ser comprensible para otro ingeniero.

Los errores clásicos de principiante

Usar la conversación como base de datos.

"Ya se lo expliqué antes."

Mal diseño.

Debe existir en documentación si importa.

Crear un `CLAUDE.md` gigantesco.

Parece prudente, pero convierte contexto útil en ruido. Anthropic recomienda mantenerlo conciso y recurrir a Skills o reglas específicas cuando el contenido no necesita estar permanentemente cargado.

21

Pegar archivos enormes.

En Claude Code normalmente es preferible darle la ruta y permitir que busque/lea lo necesario. La propia documentación recomienda referenciar archivos grandes por ruta en lugar de pegarlos íntegramente cuando el tamaño causa problemas.

30

Mantener una sesión porque "ya conoce el proyecto".

Una sesión limpia con documentación buena puede superar a una sesión contaminada con veinte decisiones obsoletas. Anthropic recomienda explícitamente limpiar entre tareas no relacionadas.

27

No registrar por qué se tomó una decisión.

Esto:

```
Usamos PostgreSQL.
```

es mucho menos valioso que:

```
ADR-001:  
PostgreSQL elegido porque necesitamos transacciones,  
constraints relacionales y consultas analíticas.  
No introducir MongoDB sin reevaluar estos requisitos.
```

Documentar el código que Claude puede descubrir solo.

Tampoco queremos esto:

```
src/  
  controllers/  
  services/  
  utils/  
  foo/  
  bar/  
  ...
```

con cientos de líneas triviales.

Anthropic aconseja reservar `CLAUDE.md` para información realmente útil, especialmente convenciones, trampas, comandos y decisiones que no sean evidentes inspeccionando el código. ²¹

Meter todos los procedimientos en instrucciones globales.

El proceso de deployment no necesita estar presente cuando diseñas un componente UI.

Skill.

No limpiar resultados enormes.

Logs de compilación, resultados de búsqueda y contenido de archivos pueden ocupar decenas de miles de tokens en una sesión de programación; Anthropic señala explícitamente que una sola sesión de exploración o debugging puede consumir rápidamente grandes cantidades de contexto. ³¹

Corregir eternamente el mismo hilo.

A veces es mejor:

```
/clear
```

y volver con:

```
Aquí está el diagnóstico confirmado.  
Ésta es la restricción.  
No uses los enfoques A y B por estos motivos.  
Implementa C.
```

que añadir la corrección número quince. La propia guía de Claude Code recomienda limpiar cuando una conversación se ha llenado de enfoques fallidos. ²⁷

El sistema que usaría contigo durante los próximos doce meses

Si fuera tu mentor durante un año, no centraría tu aprendizaje en “hacer mejores prompts”.

Lo centraría en aprender a construir **sistemas de trabajo reproducibles con agentes**.

Tu estructura base sería:

DESARROLLO PROFESIONAL

ChatGPT

|

- |— aprendizaje
- |— arquitectura
- |— investigación
- |— mentoría

|



DOCUMENTACIÓN

|



Git repository

|

- |— código
- |— tests
- |— specs
- |— ADRs
- |— CLAUDE.md
- |— Skills
- |— issues/tasks

|



Claude Code

|

- |— investigar
- |— implementar
- |— verificar
- |— documentar

No utilizaría ChatGPT y Claude Code como competidores.

Los usaría con funciones complementarias.

ChatGPT sería tu arquitecto, profesor, interlocutor y espacio de pensamiento.

Claude Code sería el ingeniero que opera directamente sobre el repositorio.

Tu estructura de proyectos

Crearía un Proyecto de ChatGPT por producto importante:

ChatGPT

Projects/

```
|— Aprendizaje IA y Software
|— App Finanzas
|— SaaS Facturación
|— Proyecto Personal X
|— Laboratorio IA
```

Dentro de **SaaS Facturación**:

```
Chats/
|— Producto y requisitos
|— Arquitectura
|— Modelo de datos
|— Autenticación
|— Stripe
|— UI
|— Testing
|— Seguridad
|— Postmortems
```

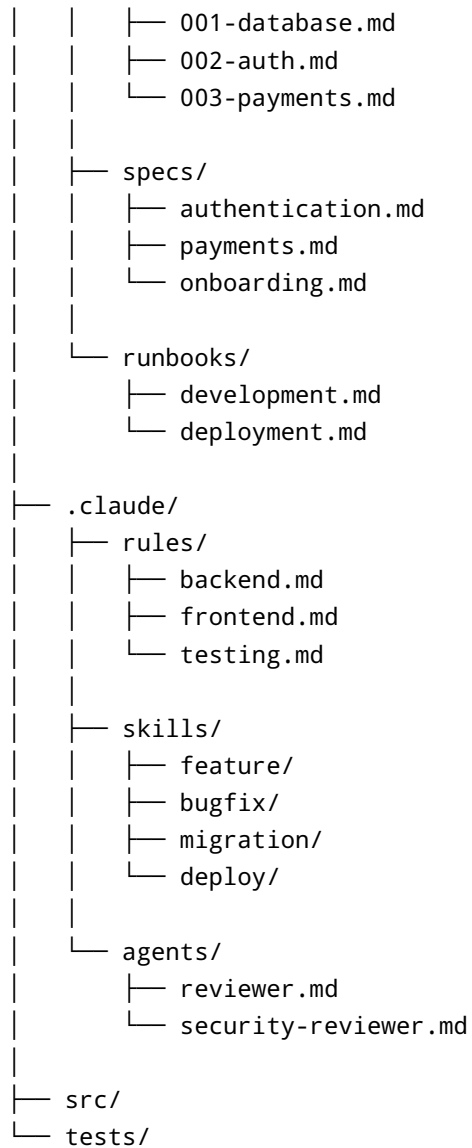
No:

```
Chat: "SaaS Facturación"
  8 meses
  19.000 mensajes
```

Los Projects permiten precisamente agrupar múltiples chats, archivos e instrucciones, y su memoria puede referenciar conversaciones relacionadas dentro del espacio. ³²

Cada repositorio tendría este esqueleto

```
project/
|
|— README.md
|— CLAUDE.md
|
|— docs/
|   |— product/
|   |   |— vision.md
|   |   |— requirements.md
|   |
|   |— architecture/
|   |   |— overview.md
|   |   |— frontend.md
|   |   |— backend.md
|   |   |— database.md
|   |
|   |— adr/
```

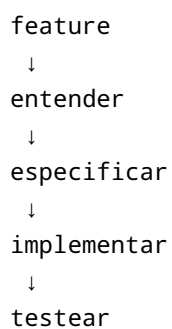


No construiría todo eso el primer día.

Evolucionaría según aparezcan necesidades.

Durante los primeros meses

Tu prioridad sería adquirir disciplina básica:



↓
commit

Usarías IA fundamentalmente para aprender.

Antes de aceptar código generado, preguntaría:

Explícame:
- qué problema resuelve,
- por qué esta arquitectura,
- qué alternativas existen,
- qué errores puede tener,
- cómo puedo comprobar que funciona.

Mantendríamos un Proyecto de ChatGPT exclusivamente de aprendizaje para que las explicaciones conceptuales no contaminen los proyectos productivos.

Tendría un archivo personal:

learning-log.md

con:

```
# Concepts learned

## HTTP
...

## Async
...

## SQL transactions
...

## Authentication
...

# Things I still don't understand
...
```

El objetivo sería que la IA aumente tu comprensión, no que oculte la complejidad.

Durante la siguiente etapa

Introduciría decisiones arquitectónicas explícitas.

Cada vez que aparezca:

“¿Usamos A o B?”

la conversación termina produciendo algo como:

```
docs/adr/007-background-jobs.md
```

No dejamos decisiones importantes enterradas en chats.

También empezaríamos a utilizar `CLAUDE.md` conscientemente.

Regla:

Quando hayas tenido que explicar la misma regla a Claude dos veces, considera documentarla.

Esto coincide con la propia recomendación de Anthropic sobre cuándo añadir información a `CLAUDE.md`: errores repetidos, correcciones recurrentes o contexto que necesitaría cualquier nuevo colaborador. ¹⁹

En la etapa intermedia

Introduciría Skills sólo después de observar tareas repetidas.

Por ejemplo, tras hacer diez funcionalidades puedes descubrir este patrón:

```
crear branch
leer spec
buscar implementación similar
implementar
crear tests
typecheck
lint
test
review diff
actualizar docs
commit
```

Entonces tiene sentido convertirlo en:

```
/feature
```

No antes.

Primero observas un proceso estable.

Después lo automatizas.

Más adelante

Introduciría subagentes para trabajos claramente aislables.

Por ejemplo:

```
Agente principal
"Implementa nueva funcionalidad"

├─ Explore
│   localiza arquitectura relevante
│
├─ security-reviewer
│   analiza riesgo
│
└─ reviewer
    revisa diff final
```

Anthropic identifica precisamente los subagentes como un mecanismo para proteger el contexto principal de grandes exploraciones y contenidos que sólo son necesarios temporalmente. ³³

Pero te obligaría a entender primero el trabajo.

Automatizar algo que todavía no entiendes sólo produce errores más deprisa.

En los últimos meses

Tu sistema empezaría a parecerse al de un pequeño equipo profesional.

La unidad de trabajo sería:

```
Issue
↓
Spec
↓
Plan
↓
Branch/worktree
↓
Claude Code session
↓
Implementation
↓
Tests
↓
Review
↓
Commit / PR
```

↓
Documentation

Y podrías ejecutar tareas independientes en sesiones o worktrees separados. Claude Code soporta sesiones independientes y Anthropic recomienda worktrees como mecanismo para trabajar en paralelo sobre ramas diferentes. ⁸

La documentación sería la interfaz entre sesiones.

Git sería el historial real.

Los tests serían la memoria ejecutable.

`CLAUDE.md` sería el onboarding.

Las Skills serían los procedimientos.

Los subagentes serían especialistas.

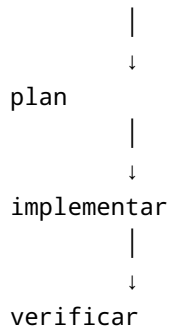
Los chats serían espacios de razonamiento.

Y ninguno intentaría sustituir a los demás.

Tu ritual para empezar una funcionalidad

Yo te pediría hacerlo siempre así:

```
¿Existe una spec?  
|  
no  
↓  
crear spec  
|  
sí  
↓  
¿hay decisiones arquitectónicas pendientes?  
|  
sí  
↓  
resolver/documentar  
|  
↓  
sesión limpia Claude Code  
|  
↓  
leer spec  
|  
↓  
explorar código
```



Prompt inicial:

Vamos a implementar docs/specs/password-reset.md.

Primero:

1. Lee CLAUDE.md.
2. Lee la especificación.
3. Explora la implementación relacionada.
4. Identifica los archivos probablemente afectados.
5. Señala cualquier contradicción entre spec y código.
6. Propón un plan.

Todavía no modifiques código.

Después:

Implementa el plan aprobado de forma incremental.

Después de cada cambio significativo:

- comprueba tipos;
- ejecuta los tests relevantes.

No modifiques cosas fuera del scope salvo que sea necesario, y en ese caso explícame por qué.

Al final:

Haz una revisión final como si fueras el reviewer de la PR.

Comprueba:

- requisitos de la spec;
- errores;
- edge cases;
- seguridad;
- tests;
- cambios innecesarios;
- documentación desactualizada.

Después muestra:

- archivos modificados;
- tests ejecutados;
- problemas restantes;
- siguiente paso.

Tu ritual al terminar una sesión

Antes de abandonar una funcionalidad:

Código coherente

✓

Tests relevantes

✓

Typecheck/lint

✓

git diff revisado

✓

Decisiones persistentes documentadas

✓

Spec actualizada si es necesario

✓

Próximo paso escrito

✓

La siguiente sesión debería poder arrancar sin preguntarte:

“¿Qué hicimos ayer?”

Puede descubrirlo.

Ése es el estándar.

La madurez final que buscaría en ti

Al principio probablemente pensarás:

“¿Cómo hago para que ChatGPT o Claude recuerde más?”

Al cabo de unos meses quiero que preguntes:

“¿Dónde debería vivir esta información?”

Y al final del año:

“¿Cómo diseño el proyecto para que cualquier agente pueda reconstruir el contexto mínimo necesario?”

Ese último nivel es el que se aproxima realmente al **context engineering profesional**.

La evolución sería:

PRINCIPIANTE

"Quiero que el chat recuerde todo."

↓

INTERMEDIO

"Hago resúmenes para no perder contexto."

↓

AVANZADO

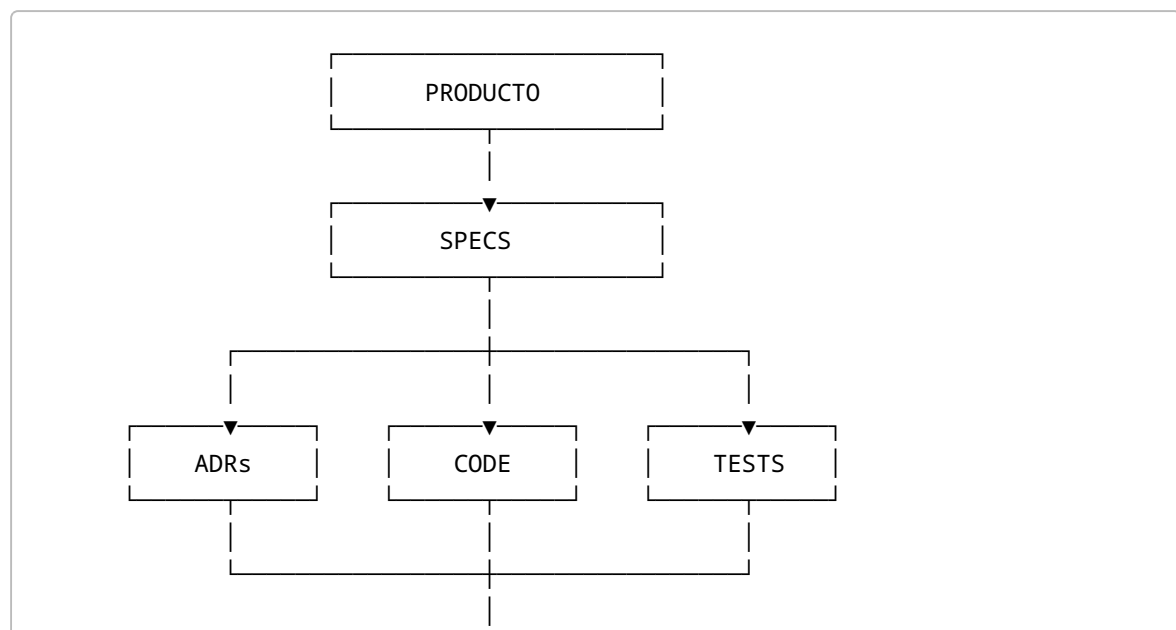
"Documento las decisiones importantes."

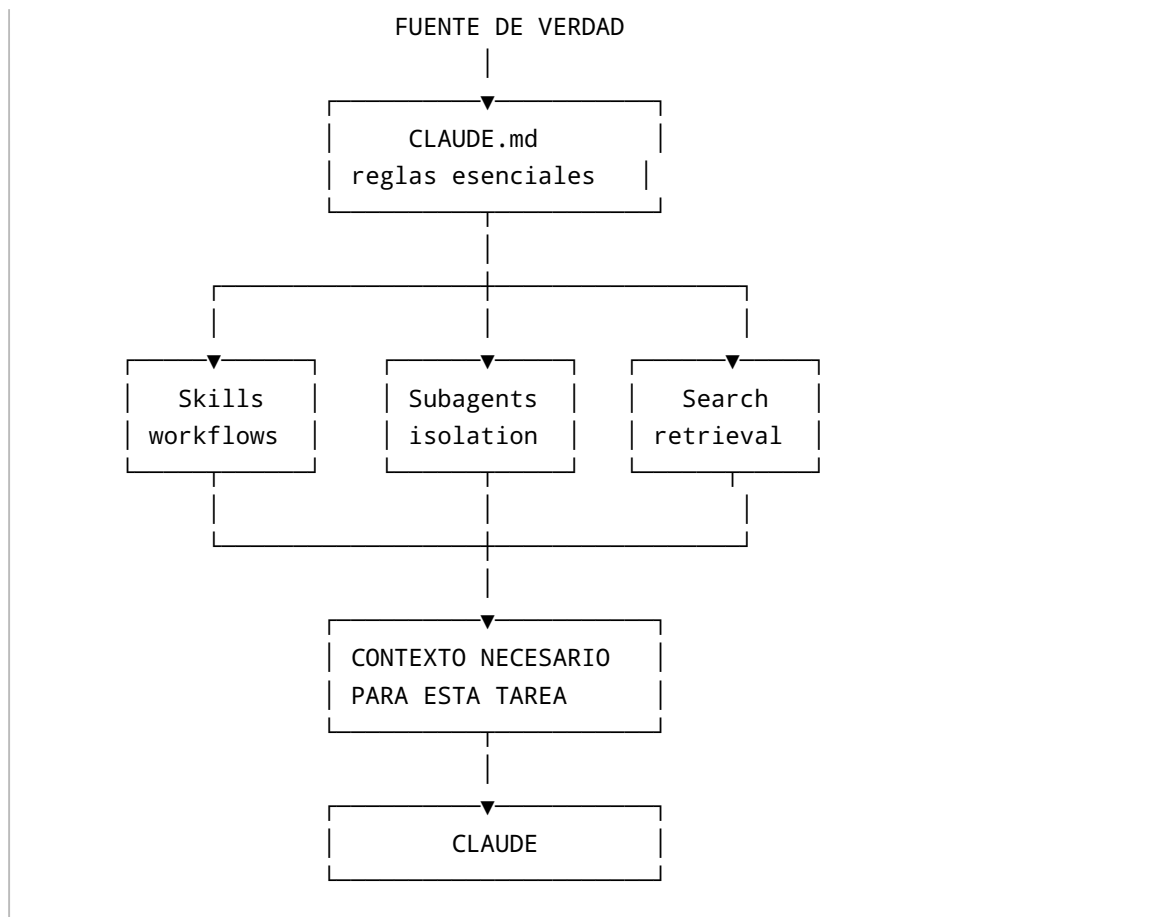
↓

PROFESIONAL

"El agente reconstruye el contexto relevante desde fuentes de verdad estructuradas."

Tu arquitectura final debería ser mentalmente ésta:





Y resumiría toda la investigación en una única regla operativa:

El chat es para pensar.

La ventana de contexto es para trabajar.

La documentación es para recordar.

Git es para saber qué ocurrió.

Los tests son para recordar cómo debe comportarse el sistema.

CLAUDE.md es para recordar cómo se trabaja en el proyecto.

Las Skills son para recordar cómo ejecutar procesos repetibles.

Los subagentes son para impedir que una investigación temporal contamine el contexto principal.

El repositorio, no la conversación, debe ser la memoria real de una aplicación que pretendas mantener durante meses o años.

Ese es el modelo mental que te recomendaría adoptar desde el primer proyecto serio. Anthropic llega a una conclusión muy similar en sus experimentos con agentes de larga duración: los sistemas fiables no intentan conservar una conversación infinita, sino dejar estados, especificaciones, tests, Git y artefactos suficientemente claros para que una nueva ventana de contexto pueda continuar el trabajo. ¹

¹ ⁹ Effective harnesses for long-running agents \ Anthropic

<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>

² ⁵ Context windows - Claude Platform Docs

https://docs.anthropic.com/en/docs/build-with-claude/context-windows?utm_source=chatgpt.com

- 3 **Attention Is All You Need**
https://arxiv.org/abs/1706.03762?utm_source=chatgpt.com
- 4 8 **How Claude Code works - Claude Code Docs**
<https://code.claude.com/docs/en/how-claude-code-works>
- 6 **Lost in the Middle: How Language Models Use Long Contexts**
https://arxiv.org/abs/2307.03172?utm_source=chatgpt.com
- 7 **Explore a janela de contexto - Claude Code Docs**
https://code.claude.com/docs/pt/context-window?utm_source=chatgpt.com
- 10 12 **Memory FAQ | OpenAI Help Center**
<https://help.openai.com/articles/8590148-memory-faq>
- 11 **How does "Reference saved memories" work?**
https://help.openai.com/en/articles/11146739-how-does-reference-saved-memories-work?utm_source=chatgpt.com
- 13 14 15 16 17 32 **Projects in ChatGPT | OpenAI Help Center**
<https://help.openai.com/en/articles/10169521-projects-in-chatgpt>
- 18 29 **Effective context engineering for AI agents \ Anthropic**
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- 19 **How Claude remembers your project - Claude Code Docs**
<https://docs.anthropic.com/en/docs/claude-code/memory>
- 20 21 26 **How Claude remembers your project - Claude Code Docs**
https://docs.anthropic.com/en/docs/claude-code/memory?utm_source=chatgpt.com
- 22 **Extend Claude with skills - Claude Code Docs**
https://docs.anthropic.com/en/docs/claude-code/skills?utm_source=chatgpt.com
- 23 **Extend Claude with skills - Claude Code Docs**
<https://docs.anthropic.com/en/docs/claude-code/skills>
- 24 25 33 **Create custom subagents - Claude Code Docs**
<https://docs.anthropic.com/en/docs/claude-code/sub-agents>
- 27 28 31 **Best practices for Claude Code - Claude Code Docs**
<https://www.anthropic.com/engineering/claude-code-best-practices>
- 30 **Error reference - Claude Code Docs**
https://code.claude.com/docs/en/errors?utm_source=chatgpt.com